

Week 4

Building a Recommender System

Contents

1	Introduction	2
1.1	The Information Overload Problem	2
2	Theoretical Foundations and Feedback Mechanisms	2
2.1	Explicit vs. Implicit Feedback	2
2.2	Formal Problem Definition	3
3	Classic Methodologies: Collaborative Filtering	3
3.1	Memory-Based Collaborative Filtering	3
3.1.1	User-Based CF	3
3.1.2	Item-Based CF	4
3.2	Model-Based CF: Matrix Factorization	4
3.2.1	Enhancements from Koren et al. (2009)	4
4	Advanced Architectures: Deep Learning in Recommendation	4
4.1	Neural Collaborative Filtering (NCF)	5
4.2	Wide & Deep Learning	5
4.3	Sequence-Aware Recommendation	5
5	Building a Recommender System: A Practical Guide	5
5.1	Step 1: Data Collection and Preprocessing	5
5.2	Step 2: Addressing the Cold-Start Problem	6
5.3	Step 3: Evaluation Metrics	6
6	Datasets and Research Resources	6
6.1	Benchmark Datasets	6
6.2	Seminal Papers and Readings	6
7	Conclusion	7

1 Introduction

Recommender systems are widely employed in industry and are ubiquitous in our daily lives. These systems are utilized in a number of areas such as online shopping sites (e.g., Amazon.com), music/movie services sites (e.g., Netflix and Spotify), mobile application stores (e.g., IOS app store and Google Play), and online advertising, just to name a few.

The major goal of recommender systems is to help users discover relevant items such as movies to watch, text to read, or products to buy, so as to create a delightful user experience. Moreover, recommender systems are among the most powerful machine learning systems that online retailers implement in order to drive incremental revenue.

Recommender systems are replacements for search engines by reducing the efforts in proactive searches and surprising users with offers they never searched for. Many companies managed to position themselves ahead of their competitors with the help of more effective recommender systems. As such, recommender systems are central to not only our everyday lives but also highly indispensable in some industries.

1.1 The Information Overload Problem

In the last decade, the Internet has evolved into a platform for large-scale online services, which profoundly changed the way we communicate, read news, buy products, and watch movies. In the meanwhile, the unprecedented number of items (we use the term item to refer to movies, news, books, and products) offered online requires a system that can help us discover items that we prefer. Recommender systems are therefore powerful information filtering tools that can facilitate personalized services and provide tailored experiences to individual users.

In short, recommender systems play a pivotal role in utilizing the wealth of data available to make choices manageable. The benefits of employing recommender systems are two-fold:

1. **User Perspective:** It can largely reduce users' effort in finding items and alleviate the issue of information overload.
2. **Business Perspective:** It can add business value to online service providers and is an important source of revenue.

Key Insight: The Dual-Value Proposition

A successful recommender system must balance the *Exploitation* of known user interests (showing them what they likely want) with the *Exploration* of new items (showing them diversity to keep engagement high).

2 Theoretical Foundations and Feedback Mechanisms

2.1 Explicit vs. Implicit Feedback

To learn the preference of users, the system shall collect feedback from them. The feedback can be either explicit or implicit.

- **Explicit Feedback:** This requires users to indicate their interests proactively. Examples include:
 - IMDb collecting star ratings ranging from one to ten stars for movies.
 - YouTube providing thumbs-up and thumbs-down buttons.

Constraint: Explicit feedback is not always readily available as many users may be reluctant to rate products.

- **Implicit Feedback:** This is often readily available since it is mainly concerned with modeling implicit behavior such as user clicks, purchase history, browsing history, watches, and even mouse movements.

– Example: A user that purchased many books by the same author probably likes that author.

Constraint: Implicit feedback is inherently noisy. We can only guess their preferences and true motives. A user watching a movie does not necessarily indicate a positive view of that movie.

2.2 Formal Problem Definition

Mathematically, the recommendation problem can be formulated as approximating a utility function $u(c, s)$ that predicts how much a user $c \in C$ will like an item $s \in S$.

$$u : C \times S \rightarrow R$$

where C is the set of users, S is the set of items, and R is a totally ordered set (e.g., non-negative integers or real numbers within a range). The objective is to choose item $s' \in S$ for user c such that:

$$s'_c = \arg \max_{s \in S} u(c, s)$$

3 Classic Methodologies: Collaborative Filtering

We start the journey with the important concept in recommender systems—collaborative filtering (CF), which was first coined by the Tapestry system. In a broad sense, it is the process of filtering for information or patterns using techniques involving collaboration among multiple users, agents, and data sources.

Overall, CF techniques can be categorized into:

1. **Memory-based CF:** Uses the entire user-item database to generate a prediction. Examples include user-based CF and item-based CF.
2. **Model-based CF:** Uses the user-item database to estimate or learn a model, which is then used for predictions. Examples include Matrix Factorization.
3. **Hybrid CF:** Combines both memory-based and model-based aspects.

3.1 Memory-Based Collaborative Filtering

Memory-based algorithms approach the collaborative filtering problem by using the database of user-item interactions to compute the similarity between users or items.

3.1.1 User-Based CF

This approach predicts a user's interest in an item based on the ratings of similar users.

Mathematical Concept: Pearson Correlation Coefficient

The similarity between user u and user v is often calculated using the Pearson correlation:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}}$$

where I_{uv} is the set of items rated by both users, $r_{u,i}$ is the rating of user u on item i , and $\bar{r}_u$ is the average rating of user u .

3.1.2 Item-Based CF

Proposed by Amazon to handle scale, this looks at the set of items the target user has rated and computes how similar they are to the target item i .

$$\text{sim}(i, j) = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\| \cdot \|\vec{j}\|}$$

Item-based CF is generally more stable than user-based CF because item characteristics (and thus the ratings they receive) tend to be more static than user preferences.

3.2 Model-Based CF: Matrix Factorization

Memory-based CF has limitations in dealing with sparse and large-scale data. Model-based methods become more popular with their better capability in dealing with sparsity and scalability.

Latent factor models, such as **Matrix Factorization (MF)**, characterize both items and users by vectors of factors inferred from item rating patterns. High correspondence between item and user factors leads to a recommendation.

3.2.1 Enhancements from Koren et al. (2009)

As detailed in "Matrix Factorization Techniques for Recommender Systems" (Koren et al., 2009), the basic dot product $q_i^T p_u$ is insufficient because it does not account for systematic biases. Some users tend to rate everything higher (optimists), and some items are naturally more popular. To address this, bias terms are added to the prediction rule:

$$\hat{r}_{ui} = \mu + b_i + b_u + q_i^T p_u$$

where:

- μ is the global average rating.
- b_i is the bias for item i (deviation from average).
- b_u is the bias for user u (deviation from average).

Mathematical Concept: SVD Optimization Objective

The optimization problem is to minimize the regularized squared error:

$$\min_{p^*, q^*, b^*} \sum_{(u, i) \in \kappa} (r_{ui} - \mu - b_i - b_u - q_i^T p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2 + b_u^2 + b_i^2)$$

This is typically solved using **Stochastic Gradient Descent (SGD)** or **Alternating Least Squares (ALS)**.

4 Advanced Architectures: Deep Learning in Recommendation

Many model-based CF approaches can be extended with neural networks, leading to more flexible and scalable models.

4.1 Neural Collaborative Filtering (NCF)

Proposed by He et al. (2017), NCF addresses a key limitation of Matrix Factorization: the fixed inner product. The inner product forces the latent space to be linear, which may not capture complex user-item interactions. NCF replaces this with a general function f :

$$\hat{y}_{ui} = f(P^T v_u^U, Q^T v_i^I)$$

The architecture fuses two sub-networks:

- **Generalized Matrix Factorization (GMF):** A neural interpretation of standard MF that uses element-wise products of user/item vectors.
- **Multi-Layer Perceptron (MLP):** Concatenates user and item vectors and passes them through dense layers (ReLU activations) to capture non-linear interactions.
- **NeuMF:** The final layer fuses GMF and MLP, allowing the model to learn both linearity and non-linearity simultaneously.

4.2 Wide & Deep Learning

Introduced by Cheng et al. (2016) at Google, this architecture solves the trade-off between **Memorization** and **Generalization**.

- **Wide Component (Memorization):** Uses a generalized linear model with cross-product transformations to learn frequent co-occurrence patterns (e.g., "User bought milk" AND "User bought bread"). It is effective for "exception rules" but fails on unseen combinations.
- **Deep Component (Generalization):** Uses embedding vectors and feed-forward neural networks to explore new feature combinations that have rarely or never occurred in the training data (e.g., inferring a user likes sci-fi based on generic embeddings, even if they haven't bought a specific sci-fi item).

4.3 Sequence-Aware Recommendation

In many real-world scenarios (e.g., session-based recommendation), the order of interactions matters.

- **RNNs (Recurrent Neural Networks):** GRU4Rec uses Gated Recurrent Units to model the session sequence.
- **Transformers (SASRec):** Self-Attentive Sequential Recommendation uses the attention mechanism to weigh the importance of past interactions differently, allowing the model to capture long-term dependencies better than RNNs.

5 Building a Recommender System: A Practical Guide

Building a production-ready recommender system involves a pipeline of distinct stages.

5.1 Step 1: Data Collection and Preprocessing

- **Data Sources:** Aggregate logs (clicks, views), transactional data (purchases), and metadata (item descriptions).
- **Handling Sparsity:** Real-world matrices are $> 99\%$ sparse. You may need to bin continuous variables or filter out users/items with very few interactions.
- **Normalization:** Normalize ratings (e.g., subtract user mean) to handle users who are "tough raters" vs. "generous raters."

5.2 Step 2: Addressing the Cold-Start Problem

New users or items have no history.

- **For New Users:** Use non-personalized popularity baselines (Top 10 Global) or demographic-based recommendations until enough data is collected.
- **For New Items:** Use Content-Based Filtering (utilizing metadata like genre, director, description) to map the new item to existing user preferences.

5.3 Step 3: Evaluation Metrics

Accuracy (RMSE) is often insufficient for ranking tasks.

- **Precision@K:** The proportion of recommended items in the top-K set that are relevant.
- **Recall@K:** The proportion of relevant items found in the top-K recommendations.
- **NDCG (Normalized Discounted Cumulative Gain):** Accounts for the position of the hit. A relevant item at rank 1 is better than at rank 10.

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

6 Datasets and Research Resources

6.1 Benchmark Datasets

Access to high-quality datasets is crucial for training and benchmarking.

Dataset	Domain	Description
MovieLens	Movies	Available in 100K, 1M, and 20M sizes. The "Hello World" of recommender systems containing explicit user ratings.
Netflix Prize	Movies	A large-scale explicit feedback dataset released during the famous competition to improve prediction accuracy.
Amazon Product	E-Commerce	Massive dataset (142 million reviews) that includes rich metadata, product descriptions, and implicit signals.
Last.fm	Music	Focuses on implicit feedback (play counts) and includes social tagging and friendship graphs.
Yelp Dataset	Local Services	Rich in text reviews, user location data, check-ins, and social graph information.

Table 1: Essential Datasets for Recommender System Research

6.2 Seminal Papers and Readings

To deepen your understanding, the following papers are highly recommended:

Resource: Foundational Papers

- **Matrix Factorization Techniques for Recommender Systems** (Koren et al., 2009) - This paper solidified the use of SVD and bias terms in collaborative filtering, winning the Netflix Prize.
- **Wide & Deep Learning for Recommender Systems** (Cheng et al., 2016) - Google's architecture that combines linear models (memorization) with deep neural networks (generalization) to improve app store recommendations.
- **Neural Collaborative Filtering** (He et al., 2017) - Introduces the NCF framework, proving that replacing the dot product with a learned neural function can significantly improve expressiveness.
- **DeepFM: A Factorization-Machine based Neural Network** (Guo et al., 2017) - An end-to-end model that learns both low-order and high-order feature interactions without any manual feature engineering.

7 Conclusion

Recommender systems have evolved from simple neighbor-based heuristics to complex deep learning architectures capable of processing massive scale, sparse data. While modern algorithms like DeepFM and SASRec offer superior accuracy, the choice of a system depends heavily on the available data type (explicit vs. implicit), the latency requirements, and the need for interpretability. Future advancements lie in *Reinforcement Learning* for long-term reward optimization and *Graph Neural Networks* for capturing high-order connectivity between users and items.